Московский государственный технический университет
имени Н. Э. Баумана

Кафедра ИУ5

Рубежный контроль 2

Рефакторинг и модульное тестирование
Вариант 14В

Дисциплина
Парадигмы и конструкции языков программирования

Выполнил: студент группы ИУ5-31Б
Летуновский Артемий Алексеевич

Проверил: Гапанюк Юрий Евгеньевич

Москва 2026

Задание и рефакторинг

Провести рефакторинг программы РК-1 и создать три модульных теста с применением фреймворка тестирования. Используется стандартный фреймворк Python unittest, который позволяет задавать ожидаемые результаты и запускать проверки независимо от консольного интерфейса.

Изменения программы

Классы перенесены в models.py, демонстрационные данные - в data.py. Запросы выделены в функции query_1, query_2 и query_3 файла queries.py. Эти функции получают списки объектов через параметры, возвращают результаты и не печатают их. Файл main.py отвечает только за подготовку демонстрационных данных и вывод.

Условия запросов сохранены: название диска на «А», минимум длительности по библиотеке и сортировка связей многие-ко-многим по дискам. Вывод исходной и переработанной программы сравнен автоматически и совпадает побайтово.

Три модульных теста

test_query_1 проверяет отбор дисков на «А» и выбор библиотеки по library_id. test_query_2 проверяет минимум, порядок библиотек и исключение пустой библиотеки. test_query_3 проверяет соединение через CDLibrary, несколько библиотек у одного диска и порядок строк.

Каждый метод дополнительно проверяет пустые входные данные. Набор тестов содержит ровно три метода test_*. Ожидаемые ответы записаны явно; тестовые объекты отличаются от демонстрационных. Подготовка данных выполняется в setUp перед каждым методом.

Чтобы проверить чувствительность тестов, временно подменялись функции: в B1 убиралась фильтрация, в B2 минимум заменялся максимумом, в B3 обращался порядок сортировки. Каждый из трех дефектов обнаружен соответствующим тестом. В итоговом коде эти дефекты отсутствуют.

Текст программы

models.py

```
"""Классы данных из РК-1 без консольного ввода и вывода."""


class CD:
    def __init__(self, id, title, duration, library_id):
        self.id = id
        self.title = title
        self.duration = duration
        self.library_id = library_id


class Library:
    def __init__(self, id, name):
        self.id = id
        self.name = name


class CDLibrary:
    def __init__(self, cd_id, library_id):
        self.cd_id = cd_id
        self.library_id = library_id
```

Текст программы

data.py

```
"""Те же демонстрационные данные, что и в РК-1."""

from models import CD, Library, CDLibrary


def sample_data():
    libraries = [
        Library(1, "Городская библиотека CD"),
        Library(2, "Архивная библиотека CD"),
        Library(3, "Фонотека"),
    ]
    cds = [
        CD(1, "Акустика", 42, 3),
        CD(2, "Рок", 55, 1),
        CD(3, "Джаз", 48, 2),
        CD(4, "Классика", 64, 1),
        CD(5, "Атмосфера", 36, 2),
    ]
    links = [
        CDLibrary(1, 1), CDLibrary(1, 3),
        CDLibrary(2, 1), CDLibrary(2, 2),
        CDLibrary(3, 2), CDLibrary(4, 3),
        CDLibrary(5, 1),
    ]
    return cds, libraries, links
```

Текст программы

queries.py

```
"""Чистые функции запросов варианта B."""


def query_1(cds, libraries):
    """Названия дисков на A и библиотеки: один-ко-многим."""
    return [
        (cd.title, library.name)
        for cd in cds
        for library in libraries
        if cd.library_id == library.id and cd.title.startswith("A")
    ]


def query_2(cds, libraries):
    """Библиотеки по минимальной длительности, по возрастанию."""
    result = []
    for library in libraries:
        durations = [
            cd.duration for cd in cds if cd.library_id == library.id
        ]
        if durations:
            result.append((library.name, min(durations)))
    return sorted(result, key=lambda row: row[1])


def query_3(cds, libraries, links):
    """Все связи многие-ко-многим с сортировкой по дискам."""
    return sorted(
        [
            (cd.title, library.name)
            for link in links
            for cd in cds
            for library in libraries
            if link.cd_id == cd.id and link.library_id == library.id
        ],
        key=lambda row: (row[0], row[1]),
    )
```

Текст программы

main.py

```
"""Консольная оболочка рефакторинга РК-1, вариант 14В."""

from data import sample_data
from queries import query_1, query_2, query_3


def main():
    cds, libraries, links = sample_data()
    results = (
        ("В1. Диски с названием на А и их библиотеки",
         query_1(cds, libraries)),
        ("В2. Библиотеки по минимальной длительности, мин",
         query_2(cds, libraries)),
        ("В3. Все связи с сортировкой по названиям дисков",
         query_3(cds, libraries, links)),
    )
    for title, rows in results:
        print(title)
        for first, second in rows:
            print(f"  {first} | {second}")
        print()


if __name__ == "__main__":
    main()
```

Текст программы

test_queries.py Часть 1 из 2

```
"""Три модульных теста: по одному на каждый запрос."""

import unittest
from models import CD, Library, CDLibrary
from queries import query_1, query_2, query_3


class QueryTests(unittest.TestCase):
    def setUp(self):
        # Данные отличаются от демонстрации РК-1.
        self.libraries = [
            Library(1, "Библиотека 1"),
            Library(2, "Библиотека 2"),
            Library(3, "Пустая библиотека"),
        ]
        self.cds = [
            CD(1, "Рок", 70, 1),
            CD(2, "Альбом", 40, 2),
            CD(3, "Джаз", 30, 1),
            CD(4, "Акустика", 50, 2),
        ]
        self.links = [
            CDLibrary(1, 2), CDLibrary(2, 1),
            CDLibrary(2, 2), CDLibrary(3, 1),
        ]

    def test_query_1(self):
        expected = [
            ("Альбом", "Библиотека 2"),
            ("Акустика", "Библиотека 2"),
        ]
        self.assertEqual(query_1(self.cds, self.libraries), expected)
        self.assertEqual(query_1([], self.libraries), [])
        self.assertEqual(query_1(self.cds, []), [])

    def test_query_2(self):
        # Минимум отличается от среднего, максимума и суммы.
        expected = [("Библиотека 1", 30), ("Библиотека 2", 40)]
        self.assertEqual(query_2(self.cds, self.libraries), expected)
        self.assertEqual(query_2([], self.libraries), [])
        self.assertEqual(query_2(self.cds, []), [])

    def test_query_3(self):
        expected = [
            ("Альбом", "Библиотека 1"),
            ("Альбом", "Библиотека 2"),
            ("Джаз", "Библиотека 1"),
            ("Рок", "Библиотека 2"),
        ]
        self.assertEqual(
```

Продолжение программы

test_queries.py Часть 2 из 2

```
            query_3(self.cds, self.libraries, self.links), expected
        )
        self.assertEqual(query_3(self.cds, self.libraries, []), [])
        self.assertEqual(query_3([], self.libraries, self.links), [])


if __name__ == "__main__":
    unittest.main()
```

Результаты проверок

Фактические результаты трех тестов и проверки намеренных дефектов

```
> python -m unittest discover -s rk2 -v
test_query_1 (test_queries.QueryTests.test_query_1) ... ok
test_query_2 (test_queries.QueryTests.test_query_2) ... ok
test_query_3 (test_queries.QueryTests.test_query_3) ... ok

----------------------------------------------------------------------
Ran 3 tests in 0.000s

OK

> Проверка намеренных дефектов
query_1: намеренный дефект обнаружен тестом
query_2: намеренный дефект обнаружен тестом
query_3: намеренный дефект обнаружен тестом
```

Все три теста unittest завершились успешно. Проверка каждого намеренного дефекта дала ожидаемое падение теста. При запуске исправной программы вывод полностью совпал с РК-1.

Запуск и вывод

Из корня репозитория: python rk2/main.py. Для запуска тестов: python -m unittest discover -s rk2 -p "test_*.py" -v. Дополнительные библиотеки не требуются.

Рефакторинг отделил вычисления от вывода и позволил проверять запросы на независимых наборах данных. Три теста проверяют отбор, агрегирование и соединение объектов, а также пустые данные. Функциональность варианта 14В сохранена.

Условия РК-2
https://github.com/ugapanyuk/courses_current/blob/main/python/rk/pcpl/pcpl_rk_2.pdf